

OpenMower-TAF-ROS

Aenderungsdokumentation

GPS-State GNSS- und RTK-Zustandsdiagnose

Ziel der Aenderung

Unter den bestehenden MQTT-Pruefungen 02_rtk_fixed und 03_gps_input_accuracy werden die aktuellen Zustaende No Fix, GPS Fix / 2D oder 3D Fix, RTK Float und RTK Fixed sowie die aktuelle Eingangsgenauigkeit verstaendlich dargestellt. Die MQTT-Feldstruktur bleibt unveraendert.

Angabe	Wert
Projektname	OpenMower-TAF-ROS
Dokumentenart	Aenderungsdokumentation
Thema	GPS-State GNSS- und RTK-Zustandsdiagnose
Version	v0.1
Status	Bereitgestellt
Erstellungsdatum	2026-07-10
Aenderungsdatum	2026-07-10
Autor / verantwortlich	OpenAI - ChatGPT
Ablageort	_Dokumentation/2026-07-10 - OpenMower-TAF-ROS - Aenderungsdokumentation - GPS-State GNSS- und RTK-Zustandsdiagnose - v0.1.pdf und .docx
Referenzen	OpenMower-TAF-ROS(1).zip; Regeln_Projektdokumentation_PDF_DOCX_Pflicht_.pdf

PDF vorrangig - DOCX bearbeitbar

1. Zusammenfassung

Die vollstaendige Variante erweitert die interne Kommunikation mit dem ZED-F9P, ohne die bestehende MQTT-Struktur zu vergroessern. Der UBX-Treiber wertet jedes UBX-NAV-PVT-Telegramm aus - auch dann, wenn die Positionsdaten fuer die Navigation ungueltig sind. Daraus entsteht ein separates, retained ROS-Diagnosetopic. xbot_monitoring verwendet diese Diagnose fuer die vorhandenen Felder value und display der State-0-Pruefungen.

- No Fix, GPS Fix / 2D oder 3D Fix, RTK Float und RTK Fixed sind eindeutig unterscheidbar.
- Die vom F9P gemeldete horizontale Genauigkeit hAcc wird weiterhin als Meterwert genutzt und zusaetzlich lesbar in Metern und Zentimetern beschrieben.
- Stufe 3 bleibt sichtbar, auch wenn Stufe 2 wegen fehlendem RTK Fixed blockiert.
- Ungueltige GNSS-Positionen werden weiterhin nicht als AbsolutePose an die Navigation uebergeben.
- Die eigentliche Fahrfreigabelogik und ihre Grenzwerte bleiben unveraendert.

1.1 Lieferumfang

Bestandteil	Inhalt
Vollstaendiges Codepaket	Gesamtes Projekt mit unveraendertem Root-Ordner OpenMower-TAF-ROS.
Geaenderte Dateien	Nur die geaenderten Dateien in ihrer Originalstruktur, einschliesslich Dokumentation.
Patch	Unified-Diff fuer die sechs geaenderten Quell- und Launch-Dateien.
Dokumentation	Inhaltsgleiche PDF- und DOCX-Ausgabe.
Commit-Nachrichten	Separate englische Commit-Nachrichten fuer das Submodul und das Hauptprojekt.

2. Ausgangslage und Problem

Vor der Aenderung wurde der RTK-Zustand im Monitoring ausschliesslich aus den RTK-Flags der gueltig publizierten AbsolutePose abgeleitet. Der Wert none war dadurch mehrdeutig: Er konnte einen normalen GPS-/3D-Fix, einen echten No-Fix-Zustand oder fehlende aktuelle Daten bedeuten.

Der F9P-Treiber erkannte den allgemeinen UBX-FixType bereits intern. Bei gnssFixOK = false oder invalidLlh = true wurde das NAV-PVT-Telegramm jedoch vor der ROS-Ausgabe verworfen. Diese Sicherheitsentscheidung ist fuer die Navigation richtig, verhinderte aber eine eindeutige Diagnose im MQTT-State.

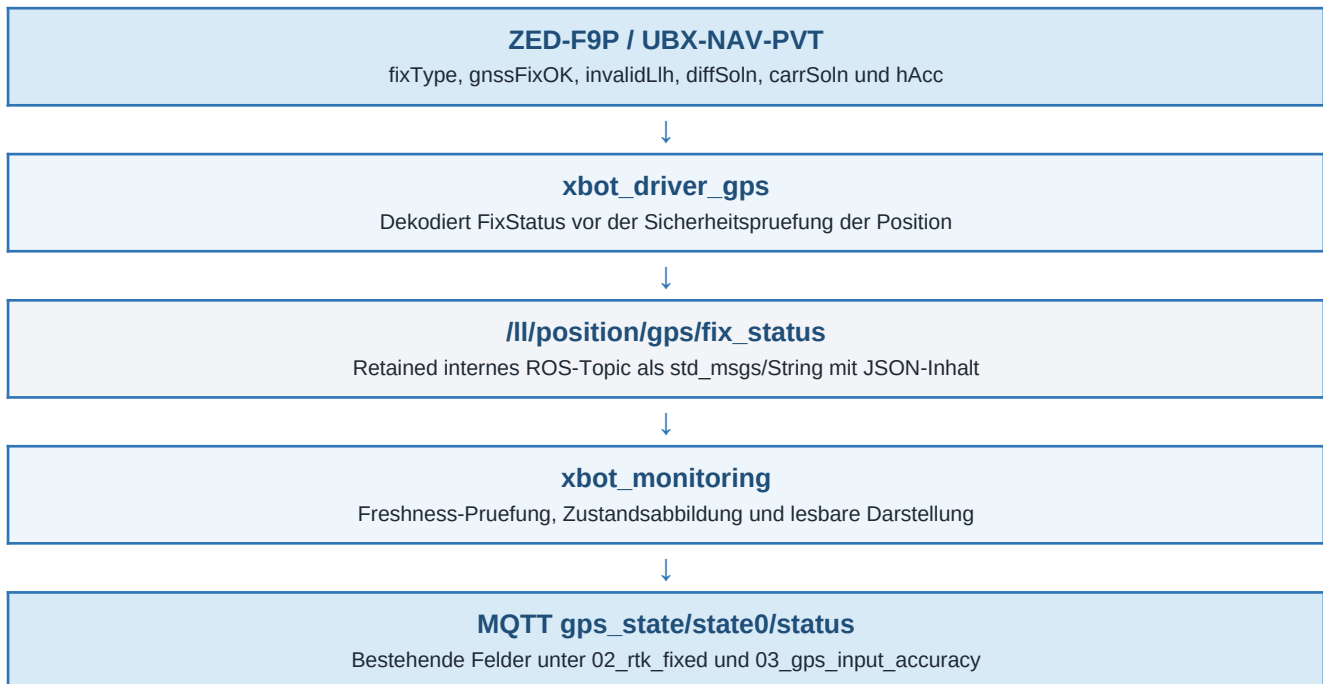
Zusaetzlich wurde Stufe 03_gps_input_accuracy bisher als nicht bewertet markiert, sobald Stufe 02_rtk_fixed blockierte. Deshalb war bei RTK Float oder normalem GPS Fix nicht sichtbar, welche Genauigkeit der Empfaenger aktuell meldete.

Nicht geaendert

RTK Fixed bleibt die notwendige Bedingung in Stufe 2. Ein guter Accuracy-Wert bei RTK Float oder GPS Fix hebt die Blockierung nicht auf. Die neue Darstellung liefert Diagnoseinformationen, aber keine neue oder lockerere Fahrfreigabe.

3. Technische Loesung

Die Loesung trennt Positionsdaten konsequent von Empfaengerdiagnose. Ein ungueltiger Fix wird nur auf dem Diagnosetopic gemeldet; der bestehende Positionskanal bleibt frei von ungueltigen Koordinaten.



3.1 Internes FixStatus-Modell

Im GPS-Interface wurde ein internes FixStatus-Objekt mit Zeitstempeln, Gueltigkeitsmerkmalen, FixType, RTKType und horizontaler Genauigkeit ergaenzt. Der Callback wird bei jedem UBX-NAV-PVT-Aufruf ausgefuehrt, bevor ein ungueltiger Positionsdatensatz verworfen wird.

Vereinfachter Ausschnitt aus `gps_interface.h`

```

struct FixStatus {
    uint32_t sensor_time;
    uint32_t received_time;
    bool gnss_fix_ok;
    bool invalid_llh;
    bool position_valid;
    double position_accuracy;
    GpsState::FixType fix_type;
    GpsState::RTKType rtk_type;
};
  
```

3.2 Internes ROS-Diagnosetopic

`driver_gps_node` publiziert den Status retained auf dem privaten Topic `~fix_status`. In `_comms.launch` wird dieses Topic nach `/ll/position/gps/fix_status` umgebogen. Das Topic ist bewusst intern; die MQTT-Nutzdaten erhalten keine neuen Felder.

Beispiel: internes ROS-Fixstatus-Telegramm

```

{
  "source": "xbot_driver_gps",
  "protocol": "UBX",
  "solution_state": "float",
  "fix_type": "3d_fix",
  "rtk_state": "float",
  "gnss_fix_ok": true,
  "invalid_llh": false,
  "position_valid": true,
  "position_accuracy_m": 0.126
}
  
```

3.3 Aktualitaet und Rueckfallverhalten

- Der Fixstatus wird nur verwendet, solange er hoechstens 3 Sekunden alt ist.
- Ist der interne Status nicht aktuell, wird fuer compatible Alt- oder NMEA-Konfigurationen auf eine aktuelle `/ll/position/gps-AbsolutePose` zurueckgefallen.
- Ein veraltetes RTK Fixed wird dadurch nicht unbegrenzt weiter angezeigt.
- Die MQTT-State-Nachrichten werden beim Wechsel des Empfaengerzustands erneut publiziert, sofern bereits ein GPS-State-Snapshot vorhanden ist.

4. Zustandsabbildung

Die folgende Tabelle zeigt, wie die UBX-NAV-PVT-Informationen auf den bestehenden MQTT-Wert 02_rtk_fixed.value abgebildet werden.

UBX-Bedingung	MQTT value	display	Bedeutung
gnssFixOK = false oder invalidLlh = true	no_fix	No Fix - keine brauchbare GNSS-Position	Der Empfaenger liefert keine fuer GNSS-Navigation gueltige Position.
fixType = DR_ONLY	no_fix	No Fix - nur Dead Reckoning	Es liegt keine eigenstaendige brauchbare GNSS-Position vor.
gueltiger 2D-/3D-/GNSS+DR-Fix, kein Carrier-Fix	gps_fix	GPS Fix / 2D oder 3D Fix - Position vorhanden, aber keine RTK-Loesung	Normale Satellitenposition ohne eindeutige RTK-Phasenloesung.
diffSoln = true und carrSoln = 1	float	RTK Float - Korrekturdaten vorhanden, Loesung noch nicht eindeutig	RTK-Korrekturen werden genutzt; Integer-Ambiguitaeten sind noch nicht geloest.
diffSoln = true und carrSoln = 2	fixed	RTK Fixed - eindeutige RTK-Loesung mit typischer Zentimetergenauigkeit	Die RTK-Mehrdeutigkeiten sind eindeutig geloest.

4.1 Accuracy-Auswertung

UBX-NAV-PVT.hAcc wird vom F9P in Millimetern geliefert und im Treiber in Meter umgerechnet. Stufe 03_gps_input_accuracy verwendet weiterhin den vorhandenen Grenzwert /xbot_positioning/max_gps_accuracy, standardmaessig 0,2 m.

- value bleibt der numerische Meterwert.
- threshold bleibt der konfigurierte Grenzwert.
- deviation bleibt value minus threshold.
- display zeigt Meter, bei Werten unter 1 m zusaetzlich Zentimeter und die Grenzwertbewertung.
- Bei No Fix wird ein eventuell vorhandener hAcc-Wert als nicht belastbar gekennzeichnet.

Typische Bereiche sind nur Orientierung

Die in der Definition genannten Bereiche - etwa 0,5 bis 3 m fuer normalen GPS Fix, 0,05 bis 0,5 m fuer RTK Float und Zentimeterwerte fuer RTK Fixed - sind keine harten Zustandsgrenzen. Fuer die Pruefung ist ausschliesslich der konfigurierte Grenzwert massgeblich.

5. MQTT-Darstellung ohne neue Felder

Die Struktur der einzelnen Live-Pruefungen bleibt unveraendert. Verwendet werden weiterhin stage, key, status, severity, value, expected beziehungsweise threshold, deviation und display. Die Bedeutung der vorhandenen Inhalte wurde erweitert; definition_version wurde deshalb von 1 auf 2 angehoben.

5.1 RTK Fixed

```
"02_rtk_fixed": {
  "stage": 2,
  "key": "rtk_fixed",
  "status": "ok",
  "severity": 0,
  "value": "fixed",
  "expected": "fixed",
  "display": "RTK Fixed - eindeutige RTK-Loesung mit typischer Zentimetergenauigkeit"
}
```

5.2 RTK Float mit sichtbarer Accuracy

```
"02_rtk_fixed": {
  "stage": 2,
  "key": "rtk_fixed",
  "status": "blocked",
  "severity": 3,
  "value": "float",
  "expected": "fixed",
  "display": "RTK Float - Korrekturdaten vorhanden, Loesung noch nicht eindeutig"
},
"03_gps_input_accuracy": {
  "stage": 3,
  "key": "gps_input_accuracy",
  "status": "ok",
  "severity": 0,
  "value": 0.126,
  "threshold": 0.2,
  "deviation": -0.074,
  "display": "0.126 m (12.6 cm) - ausreichend; Grenzwert <= 0.200 m"
}
```

5.3 GPS Fix / 3D Fix

```
"02_rtk_fixed": {
  "status": "blocked",
  "value": "gps_fix",
  "expected": "fixed",
  "display": "GPS Fix / 3D Fix - Position vorhanden, aber keine RTK-Loesung"
},
"03_gps_input_accuracy": {
  "status": "blocked",
  "value": 1.250,
  "threshold": 0.2,
  "deviation": 1.050,
  "display": "1.250 m - Grenzwert ueberschritten; Grenzwert <= 0.200 m"
}
```

5.4 No Fix

```
"02_rtk_fixed": {
  "status": "blocked",
  "value": "no_fix",
  "expected": "fixed",
  "display": "No Fix - keine brauchbare GNSS-Position"
},
"03_gps_input_accuracy": {
  "status": "blocked",
  "value": null,
  "threshold": 0.2,
  "display": "Keine aktuelle GPS-Eingangsgenauigkeit vorhanden"
}
```

Reihenfolge der Entscheidung

Stufe 2 bleibt der erste blockierende Grund, wenn RTK Fixed fehlt. Stufe 3 wird trotzdem informativ bewertet und angezeigt. Die nachfolgenden Stufen bleiben wie bisher inaktiv, solange eine fruehere Blockierung besteht.

6. Geaenderte Dateien

Datei	Aenderung
src/lib/xbot_driver_gps/src/interfaces/gps_interface.h	FixStatus-Struktur, Callback-Typ, Setter und Callback-Member ergaenz.
src/lib/xbot_driver_gps/src/interfaces/gps_interface.cpp	FixStatus-Callback initialisiert und Setter implementiert; <cmath> explizit eingebunden.
src/lib/xbot_driver_gps/src/interfaces/ublox_gps_interface.cpp	NAV-PVT-Fix- und RTK-Zustand vor dem Verwerfen ungueltiger Positionen dekodiert; FixStatus bei jedem NAV-PVT gemeldet.
src/lib/xbot_driver_gps/src/driver_gps_node.cpp	Retained ~fix_status-Topic und lesbare Zustandsabbildung implementiert.
src/open_mower/launch/include/_comms.launch	~fix_status nach /ll/position/gps/fix_status remapped.
src/lib/xbot_monitoring/src/xbot_monitoring.cpp	Internes Fixstatus-Topic ausgewertet; eindeutige State-Werte und Accuracy-Darstellung in bestehende MQTT-Felder integriert; Definition auf Version 2 gesetzt.

6.1 Bewusst nicht geaendert

- xbot_msgs/AbsolutePose.msg wurde nicht erweitert.
- Die Positionierungs- und Mower-Logic-Grenzwerte wurden nicht veraendert.
- Es wurden keine neuen MQTT-Keys unter state0/status angelegt.
- Die Fahrfreigabe wird weiterhin aus der von xbot_positioning gelieferten xb_pose abgeleitet.
- Ein ungueltiges NAV-PVT erzeugt weiterhin keine AbsolutePose.

7. Validierung

Die Auslieferung wurde in der verfuegbaren Containerumgebung mit statischen und zielgerichteten Pruefungen kontrolliert.

Pruefung	Ergebnis
C++ Syntax: gps_interface.h	Erfolgreich mit einem kleinen Compile-Harness geprueft.
C++ Syntax: gps_interface.cpp	Erfolgreich mit g++ -std=c++17 -fsyntax-only geprueft.
C++ Syntax: ublox_gps_interface.cpp	Erfolgreich mit g++ -std=c++17 -fsyntax-only und minimalen Abhaengigkeits-Stubs geprueft.
Launch-XML	_comms.launch erfolgreich mit XML-Parser validiert.
Patch	Dry-Run und exakte Anwendung auf die hochgeladenen Originaldateien erfolgreich.
Dateiformat	CRLF der bestehenden GPS-Submoduldateien und LF von xbot_monitoring beibehalten; keine nachlaufenden Leerzeichen.
Vollstaendiger ROS-/Catkin-Build	Nicht ausgefuehrt, da in der Bearbeitungsumgebung keine ROS-Noetic-/Catkin-Toolchain vorhanden ist.

Verbleibende Laufzeitpruefung

Vor dem produktiven Einsatz ist ein Build im regulaeren OpenMower-ROS-Image und ein Hardwaretest mit dem F9P erforderlich. Die statischen Pruefungen ersetzen keinen Test mit realen NAV-PVT-, Korrektur- und MQTT-Daten.

7.1 Empfohlener Hardwaretest

1. ROS starten und pruefen, ob /ll/position/gps/fix_status regelmaessig publiziert wird.
2. Korrekturdaten deaktivieren und den Wechsel zu gps_fix beziehungsweise no_fix beobachten.
3. Korrekturdaten aktivieren und die Folge gps_fix -> float -> fixed beobachten.
4. Bei jedem Zustand gps_state/state0/status kontrollieren; Stufe 3 muss auch bei float und gps_fix einen Wert anzeigen, sofern hAcc gueltig ist.
5. Antenne kurz abschirmen und sicherstellen, dass no_fix diagnostiziert wird, ohne dass ungueltige AbsolutePose-Nachrichten entstehen.
6. Fahrfreigabe kontrollieren: Nur die bereits vorhandenen Kriterien duerfen sie freigeben.

Beispielbefehle fuer die Laufzeitpruefung

```
rostopic echo /ll/position/gps/fix_status
rostopic echo /ll/position/gps
mosquitto_sub -h <broker> -v -t 'gps_state/state0/status'
mosquitto_sub -h <broker> -v -t 'gps_state/state1'
mosquitto_sub -h <broker> -v -t 'gps_state/state2'
```

8. Installation und Commit-Reihenfolge

xbot_driver_gps ist ein eigenes Git-Submodul. Deshalb muessen die GPS-Treiberdateien zuerst im Submodul committed und gepusht werden. Danach wird im Hauptprojekt der neue Submodulzeiger gemeinsam mit Monitoring, Launch-Datei und Dokumentation committed.

8.1 Submodul

```
cd OpenMower-TAF-ROS/src/lib/xbot_driver_gps
git add src/interfaces/gps_interface.h \
        src/interfaces/gps_interface.cpp \
        src/interfaces/ublox_gps_interface.cpp \
        src/driver_gps_node.cpp
git commit -F ../../../../Dokumentation/*xbot_driver_gps*GNSS-Fixdiagnose*.txt
git push
```

8.2 Hauptprojekt

```
cd ../../../../
git add src/lib/xbot_driver_gps \
        src/lib/xbot_monitoring/src/xbot_monitoring.cpp \
        src/open_mower/launch/include/_comms.launch \
        _Dokumentation
git commit -F _Dokumentation/*Hauptprojekt*GPS-State*GNSS-Diagnose*.txt
git push
```

Wichtig beim Submodul

Der Hauptprojekt-Commit darf erst erstellt werden, nachdem der neue xbot_driver_gps-Commit im Submodul existiert. Andernfalls verweist das Hauptprojekt nicht reproduzierbar auf den angepassten Treiberstand.

9. Rueckbau

Der Rueckbau erfolgt in umgekehrter Reihenfolge: Zuerst wird im Hauptprojekt auf den vorherigen Submodulstand und die vorherigen Versionen von xbot_monitoring.cpp sowie _comms.launch zurueckgegangen. Anschliessend kann der Submodul-Commit bei Bedarf separat revertiert werden.

- Hauptprojekt-Commit mit git revert <commit> rueckgaengig machen.
- Im Submodul den Treiber-Commit mit git revert <commit> rueckgaengig machen, falls der Commit nicht mehr benoetigt wird.
- ROS-Image neu bauen und gps_state/state0/definition pruefen; nach Rueckbau ist wieder definition_version 1 zu erwarten.
- Retained MQTT-Nachrichten gegebenenfalls durch einen Neustart oder gezielte Neupublikation aktualisieren.

10. Aenderungshistorie

Version	Datum	Bearbeiter	Status	Aenderung
v0.1	2026-07-10	OpenAI - ChatGPT	Bereitgestellt	Erstimplementierung der vollstaendigen GNSS-/RTK-Zustandsdiagnose mit bestehender MQTT-Feldstruktur.

Anhang A - Commit-Nachricht xbot_driver_gps

BL_Publish GNSS fix diagnostics
Date: 2026-07-10

Summary:

- Decode UBX-NAV-PVT GNSS fix and RTK solution states before invalid positions are discarded.
- Publish a retained internal fix-status message for No Fix, GPS Fix, RTK Float, and RTK Fixed.
- Preserve the existing safety rule that invalid positions are never published as AbsolutePose updates.

Changed files:

- src/interfaces/gps_interface.h
- src/interfaces/gps_interface.cpp
- src/interfaces/ublox_gps_interface.cpp
- src/driver_gps_node.cpp

Anhang B - Commit-Nachricht Hauptprojekt

BS_Show GNSS solution details in GPS state
Date: 2026-07-10

Summary:

- Remap the retained internal GNSS fix-status topic from xbot_driver_gps.
- Distinguish No Fix, GPS Fix / 2D or 3D Fix, RTK Float, and RTK Fixed in the existing MQTT gps_state fields.
- Keep the current GPS input accuracy visible in stage 3 even when RTK Fixed is not reached.
- Keep the MQTT field structure unchanged and update the state0 definition version to 2.
- Add the PDF and DOCX implementation documentation, source patch, and commit messages.

Ende der Dokumentation